Contents

TP Jour 1 : Opérer et factoriser le CI de TaskFlow **1**
Prérequis . . . 1
Étape 1 : Créer le repo et enregistrer le runner self-hosted (45 min) . . . 2
1.1 Copier le starter et créer le repo `taskflow-ops` . . . 2
1.2 Vérifier l'application en local (facultatif) . . . 2
1.3 Obtenir un token d'enregistrement de runner . . . 2
1.4 Lancer le runner en conteneur . . . 2
1.5 Premier job sur le runner . . . 3
Étape 2 : Action composite `setup-node-project` (30 min) . . . 4
2.1 Créer l'action . . . 4
2.2 L'utiliser dans `hello-runner.yml` pour la tester . . . 5
Étape 3 : Workflow réutilisable `reusable-node-ci.yml` (1h15) . . . 5
3.1 Créer le workflow appelable . . . 5
3.2 Remplacer `ci.yml` par deux appels . . . 8
3.3 Lire les rapports . . . 9
Étape 4 : Optimiser, casser, protéger (45 min) . . . 9
4.1 Ignorer les changements de documentation . . . 9
4.2 Casser un test volontairement . . . 9
4.3 Vérifier `concurrency` . . . 10
4.4 Protéger `main` avec des checks requis . . . 10
Étape 5 (bonus) : Audit, matrix et badge (30 min) . . . 10
5.1 Job d'audit de sécurité . . . 10
5.2 Matrix de versions Node pilotée par input . . . 11
5.3 Badge de statut . . . 11
Livrable & critères de validation . . . 11
Checklist . . . 11
Critères notés (Bloc 1 — 25 points) . . . 11
Erreurs courantes . . . 12
Ressources . . . 12

TP Jour 1 : Opérer et factoriser le CI de TaskFlow

> **Durée** : ~3h45 | **Objectif** : Installer et enregistrer un runner self-hosted GitHub Actions, puis refactoriser le CI de TaskFlow en **workflow réutilisable** + **action composite**, appelés pour le front et pour l'API, avec rapports de tests JUnit, résumé de couverture, `concurrency` et checks requis sur `main`.

Prérequis

- Docker + Docker Compose v2 fonctionnels (`docker compose version`)
- Un compte GitHub (le CI tourne dans **votre** repo)
- `git` et, idéalement, `gh` (GitHub CLI) installés et authentifiés
- Node 20 en local pour vérifier avant de pousser (facultatif mais recommandé)

Consultez `../../ressources/setup-lab.md` si l'un de ces outils n'est pas encore installé.

Étape 1 : Créer le repo et enregistrer le runner self-hosted (45 min)

1.1 Copier le starter et créer le repo `taskflow-ops`

```
cp -r tp/fil-rouge-taskflow/starter/ ~/taskflow-ops
cd ~/taskflow-ops
git init -b main
git add .
git commit -m "chore: starter TaskFlow opérée"
```

Créez le repo sur GitHub (public ou privé, à votre convenance) puis poussez :

```
# Avec gh :
gh repo create taskflow-ops --private --source=. --remote=origin --push
# Ou à la main :
# git remote add origin git@github.com:<vous>/taskflow-ops.git && git push -u origin
```

Résultat attendu : l'onglet *Actions* du repo montre un premier run du workflow `CI` hérité du cours précédent (lint / test / build du front). Il doit être **vert**.

> Ce `ci.yml` est celui que vous aviez écrit dans le cours *Initialisation CI*. Aujourd'hui vous allez le **factoriser** : il ne teste que le front, alors que le projet contient maintenant aussi une API dans `api/`.

1.2 Vérifier l'application en local (facultatif)

```
npm ci && npm run test:ci && npm run build
cd api && npm ci && npm run test:ci && cd ..
```

Résultat attendu : les deux suites passent, et deux fichiers `reports/junit.xml` (racine et `api/`) ont été générés. C'est ce format XML que le CI publiera.

1.3 Obtenir un token d'enregistrement de runner

Dans votre repo GitHub : **Settings → Actions → Runners → New self-hosted runner**. Choisissez *Linux / x64* : la page affiche les commandes `config.sh` et un **token** (`A…`, valable 1 heure).

> Nous n'exécutons pas `config.sh` à la main : le lab fournit un conteneur qui fait exactement la même chose (téléchargement de l'agent, `config.sh`, `run.sh`). Copiez seulement le token.

1.4 Lancer le runner en conteneur

```
cd <repo de la formation>/ressources/lab/runner
cp .env.example .env
```

Éditez `.env` :

```
REPO_URL=https://github.com/<vous>/taskflow-ops
RUNNER_TOKEN=AXXXXXXXXXXXXXXXXXXXXXXXXXXXX
RUNNER_NAME=lab-runner
LABELS=self-hosted,lab,ansible
```

Puis :

```
docker compose up -d --build
docker compose logs -f runner
```

Résultat attendu :

```
runner  | Configuring runner...
runner  | √ Connected to GitHub
runner  | √ Runner successfully added
runner  | √ Settings Saved.
runner  | √ Connected to GitHub
runner  | Current runner version: '2.3xx.x'
runner  | Listening for Jobs
```

Dans **Settings → Actions → Runners**, `lab-runner` apparaît avec le statut **Idle** et les labels `self-hosted`, `linux`, `x64`, `lab`, `ansible`.

> **Token expiré ?** Le token n'est valable qu'une heure et à usage unique. Regénérez-en un dans l'UI, mettez à jour `.env`, puis `docker compose down -v && docker compose up -d`.

1.5 Premier job sur le runner

Créez `.github/workflows/hello-runner.yml` :

```yaml
# Premier workflow exécuté sur NOTRE runner (et non sur ubuntu-latest chez GitHub)
name: Hello runner

on:
  workflow_dispatch:

jobs:
  hello:
    # Les labels sélectionnent le runner : self-hosted ET lab
    runs-on: [self-hosted, lab]
    steps:
      - name: Qui suis-je ?
        run: |
          echo "Runner : $RUNNER_NAME sur $(hostname)"
          echo "Utilisateur : $(whoami)"
          uname -a

      - name: Outils disponibles sur le runner
        run: |
          ansible --version | head -1
          docker --version
```

```
          node --version || echo "Node absent (normal : on utilisera setup-node)"
```

```
git add .github/workflows/hello-runner.yml
git commit -m "ci: premier workflow sur le runner self-hosted"
git push
gh workflow run hello-runner.yml
gh run watch
```

Résultat attendu : le job s'exécute sur `lab-runner` (visible dans le résumé du run : *Runner name*). Les logs du conteneur montrent `Running job: hello` puis `Job hello completed with result: Succeeded`.

> Un runner self-hosted **ne nettoie pas** son workspace entre deux runs (contrairement aux runners GitHub qui sont jetables). Gardez ce point en tête pour le J2 : on y ajoutera des étapes de nettoyage des fichiers sensibles.

Étape 2 : Action composite `setup-node-project` (30 min)

Les jobs du front et de l'API répètent les mêmes steps : `setup-node`, cache npm, `npm ci`. Une **action composite** factorise ces steps.

2.1 Créer l'action

Créez `.github/actions/setup-node-project/action.yml` :

```yaml
# Action composite : prépare un projet Node (Node + cache npm + npm ci)
# Utilisation : uses: ./.github/actions/setup-node-project
#               with: { working-directory: api }
name: Setup Node project
description: Installe Node.js, restaure le cache npm et exécute npm ci dans le répert

inputs:
  working-directory:
    description: Répertoire contenant package.json
    required: false
    default: '.'
  node-version:
    description: Version de Node.js
    required: false
    default: '20'

runs:
  using: composite
  steps:
    - name: Setup Node.js ${{ inputs.node-version }}
      uses: actions/setup-node@v4
      with:
        node-version: ${{ inputs.node-version }}
        cache: npm
```

```
        # Le cache est indexé sur le lockfile du bon répertoire
        cache-dependency-path: ${{ inputs.working-directory }}/package-lock.json

    - name: Installation des dépendances (npm ci)
      # shell est OBLIGATOIRE pour un step run dans une action composite
      shell: bash
      working-directory: ${{ inputs.working-directory }}
      run: npm ci
```

2.2 L'utiliser dans `hello-runner.yml` pour la tester

Ajoutez un job à `hello-runner.yml` :

```
  smoke-action:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: ./.github/actions/setup-node-project
        with:
          working-directory: api
      - run: node --version && ls api/node_modules | head -3
```

Résultat attendu : le job `smoke-action` est vert ; le log de `setup-node` indique `Cache not found` au premier run puis `Cache restored from key: node-cache-Linux-x64-npm-…` au second.

> **`actions/checkout` d'abord.** Une action locale (`./.github/actions/...`) n'existe dans le workspace qu'après le checkout. Sans lui : `Can't find 'action.yml'`.

Étape 3 : Workflow réutilisable `reusable-node-ci.yml` (1h15)

3.1 Créer le workflow appelable

Créez `.github/workflows/reusable-node-ci.yml` :

```
# ================================================================================
# Workflow RÉUTILISABLE : CI d'un projet Node (lint → test → build)
# Appelé par ci.yml pour le front (.) et pour l'API (api/).
# ================================================================================
name: Reusable Node CI

on:
  workflow_call:
    inputs:
      working-directory:
        description: Répertoire du projet Node
        type: string
        required: true
```

```yaml
      node-version:
        description: Version de Node.js
        type: string
        default: '20'
      upload-artifact:
        description: Publier le build (dist/) en artefact
        type: boolean
        default: false
      artifact-name:
        description: Suffixe des artefacts (dist-<nom>, coverage-<nom>)
        type: string
        default: project

jobs:
  lint:
    name:  Lint
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
      - uses: ./.github/actions/setup-node-project
        with:
          working-directory: ${{ inputs.working-directory }}
          node-version: ${{ inputs.node-version }}
      - name: ESLint
        working-directory: ${{ inputs.working-directory }}
        run: npm run lint
      - name: Prettier
        working-directory: ${{ inputs.working-directory }}
        run: npm run format:check

  test:
    name:  Tests
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
      - uses: ./.github/actions/setup-node-project
        with:
          working-directory: ${{ inputs.working-directory }}
          node-version: ${{ inputs.node-version }}

      - name: Tests + rapport JUnit + couverture
        working-directory: ${{ inputs.working-directory }}
        run: npm run test:ci

      - name: Publier le rapport de tests
        uses: dorny/test-reporter@v1
        if: always()          # même si les tests échouent : c'est justement là qu'or
```

```
        with:
          name: Tests ${{ inputs.artifact-name }}
          path: ${{ inputs.working-directory }}/reports/junit.xml
          reporter: java-junit

      - name: Résumé de couverture
        if: always()
        working-directory: ${{ inputs.working-directory }}
        run: |
          echo "## Couverture — ${{ inputs.artifact-name }}" >> "$GITHUB_STEP_SUMMARY
          echo '```' >> "$GITHUB_STEP_SUMMARY"
          # lcov.info : on calcule le % de lignes couvertes (LH = lignes touchées, LF
          awk -F: '/^LH:/{h+=$2} /^LF:/{f+=$2} END{printf "Lignes couvertes : %d/%d (
            | tee -a "$GITHUB_STEP_SUMMARY"
          echo '```' >> "$GITHUB_STEP_SUMMARY"

      - name: Archiver la couverture
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: coverage-${{ inputs.artifact-name }}
          path: ${{ inputs.working-directory }}/coverage/
          retention-days: 7

  build:
    name:  Build
    runs-on: ubuntu-latest
    needs: [lint, test]
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
      - uses: ./.github/actions/setup-node-project
        with:
          working-directory: ${{ inputs.working-directory }}
          node-version: ${{ inputs.node-version }}
      - name: Build
        working-directory: ${{ inputs.working-directory }}
        run: npm run build
      - name: Archiver le build
        if: inputs.upload-artifact
        uses: actions/upload-artifact@v4
        with:
          name: dist-${{ inputs.artifact-name }}
          path: ${{ inputs.working-directory }}/dist/
          retention-days: 7
```

L'API n'a pas de « build » à proprement parler : son script `npm run build` ne fait qu'une vérification de syntaxe (`node --check`). Le job existe quand même : c'est le **même contrat** pour tous les projets Node, c'est le principe d'un workflow

réutilisable.

3.2 Remplacer `ci.yml` par deux appels

Remplacez intégralement `.github/workflows/ci.yml` :

```yaml
# ============================================================================
# CI TaskFlow — orchestration : appelle le workflow réutilisable
# pour le front (.) et pour l'API (api/).
# ============================================================================
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

# Requis par dorny/test-reporter (crée un "check" sur le commit)
permissions:
  contents: read
  checks: write
  pull-requests: write

# Un push sur la même branche annule le run précédent encore en cours
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  front:
    name: Front
    uses: ./.github/workflows/reusable-node-ci.yml
    with:
      working-directory: .
      upload-artifact: true
      artifact-name: front

  api:
    name: API
    uses: ./.github/workflows/reusable-node-ci.yml
    with:
      working-directory: api
      artifact-name: api
```

```
git add .github
git commit -m "ci: workflow réutilisable + action composite (front + api)"
git push
gh run watch
```

Résultat attendu : le run CI affiche 6 jobs organisés en deux groupes — Front / Lint,

`Front / 🧪 Tests`, `Front / 🏗 Build`, `API / …`. L'onglet *Summary* montre deux blocs « Couverture » et deux artefacts `dist-front` et `coverage-…`. Les checks « Tests front » et « Tests api » apparaissent sur le commit.

> **`permissions` se déclare dans le workflow appelant.** Un workflow réutilisable hérite des permissions de l'appelant ; si `checks: write` manque, `dorny/test-reporter` échoue avec `Resource not accessible by integration`.

3.3 Lire les rapports

Ouvrez le run → job `Front / 🧪 Tests` → step *Publier le rapport de tests* : le lien mène à une page listant chaque test avec son temps d'exécution. Ouvrez aussi le *Summary* du run : les deux résumés de couverture y figurent.

Étape 4 : Optimiser, casser, protéger (45 min)

4.1 Ignorer les changements de documentation

Dans `ci.yml`, ajoutez des filtres pour ne pas relancer 6 jobs quand seul le README change :

```yaml
on:
  push:
    branches: [main]
    paths-ignore:
      - '**.md'
      - 'docs/**'
  pull_request:
    branches: [main]
    paths-ignore:
      - '**.md'
      - 'docs/**'
```

Testez : modifiez le `README.md`, poussez → **aucun** run `CI` ne démarre.

> Attention avec les checks requis (4.4) : un commit qui ne déclenche pas le CI n'aura pas de check… GitHub le considère comme « en attente » sur une PR. Une alternative est de laisser le workflow tourner mais de faire des jobs conditionnels avec `dorny/paths-filter`. Pour ce TP, `paths-ignore` suffit.

4.2 Casser un test volontairement

Dans `tests/tasks.test.js`, modifiez une assertion pour la rendre fausse, par exemple :

```
expect(task.priority).toBe('high') // au lieu de 'medium'
```

```
git commit -am "test: casse volontaire pour lire le rapport" && git push && gh run wa
```

Résultat attendu : `Front / 🧪 Tests` est rouge, `Front / 🏗 Build` est **skipped** (grâce à `needs`), mais le rapport de tests est quand même publié (`if: always()`) : cliquez dessus, le test en échec est listé avec le message `expected 'medium' to be 'high'`. L'API, elle, reste verte : les deux projets sont indépendants.

Réparez l'assertion et poussez à nouveau.

4.3 Vérifier `concurrency`

Poussez deux commits à quelques secondes d'intervalle :

```
git commit --allow-empty -m "ci: test concurrency 1" && git push
git commit --allow-empty -m "ci: test concurrency 2" && git push
```

Résultat attendu : le premier run passe en **Cancelled** dès que le second démarre. Sans `concurrency`, les deux auraient consommé 6 jobs chacun.

4.4 Protéger `main` avec des checks requis

Settings → Rules → Rulesets → New branch ruleset (ou *Branches → Add branch protection rule*) :

- Target : `main`
- ☑ *Require a pull request before merging* (0 approbation suffit pour le TP)
- ☑ *Require status checks to pass* → ajoutez `Front / 🏗 Build` et `API / 🏗 Build`

Vérifiez en créant une PR depuis une branche :

```
git switch -c feat/protection-test
echo "// test" >> src/app.js
git commit -am "feat: test de la protection" && git push -u origin feat/protection-te
gh pr create --fill
```

Résultat attendu : la PR affiche « *Merging is blocked* » tant que les checks ne sont pas verts, puis « *All checks have passed* » ; le bouton *Merge* devient actif. Fusionnez, supprimez la branche.

> En équipe, c'est **cette** règle qui rend le CI utile : personne ne peut fusionner du code qui ne passe pas les tests, même en cas d'urgence. Le CI cesse d'être informatif pour devenir bloquant.

Étape 5 (bonus) : Audit, matrix et badge (30 min)

5.1 Job d'audit de sécurité

Dans `reusable-node-ci.yml`, ajoutez un job :

```
  audit:
    name: 🏗 Audit
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: npm audit (vulnérabilités high/critical)
        working-directory: ${{ inputs.working-directory }}
        # Commencez avec "|| true" pour voir le rapport sans casser le build,
        # puis retirez-le une fois les dépendances à jour : l'audit devient bloquant.
        run: npm audit --audit-level=high
```

5.2 Matrix de versions Node pilotée par input

Ajoutez un input `node-versions` (type `string`, défaut `'["20"]'`) et, sur le job `test` :

```yaml
    strategy:
      fail-fast: false
      matrix:
        node: ${{ fromJSON(inputs.node-versions) }}
```

puis passez `node-version: ${{ matrix.node }}` à l'action composite et suffixez le nom du rapport : `name: Tests ${{ inputs.artifact-name }} (Node ${{ matrix.node }})`. Dans `ci.yml`, appelez le front avec `node-versions: '["20", "22"]'`.

> Pensez à suffixer aussi le **nom des artefacts** (`coverage-${{ inputs.artifact-name }}-${{ matrix.node }}`) : deux jobs ne peuvent pas uploader un artefact du même nom.

5.3 Badge de statut

Dans le `README.md` de `taskflow-ops` :

```
![CI](https://github.com/<vous>/taskflow-ops/actions/workflows/ci.yml/badge.svg)
```

Livrable & critères de validation

Livrable : le repo `taskflow-ops` poussé sur GitHub, avec un run `CI` vert sur `main`.

Checklist

- ☐ Le runner `lab-runner` est **Idle** dans *Settings → Actions → Runners* et le workflow `hello-runner.yml` a tourné dessus
- ☐ `.github/actions/setup-node-project/action.yml` existe et est utilisé par tous les jobs du workflow réutilisable
- ☐ `.github/workflows/reusable-node-ci.yml` déclare `on: workflow_call` avec les inputs `working-directory`, `node-version`, `upload-artifact`, `artifact-name`
- ☐ `ci.yml` appelle le workflow réutilisable **deux fois** (front et `api/`) et déclare `permissions` + `concurrency`
- ☐ Le rapport de tests JUnit est publié (check « Tests front » / « Tests api » sur le commit) et le résumé de couverture apparaît dans le *Summary*
- ☐ L'artefact `dist-front` est téléchargeable depuis le run
- ☐ `main` est protégée : checks `Front / Build` et `API / Build` requis

Critères notés (Bloc 1 — 25 points)

Critère	Points
Runner self-hosted enregistré et fonctionnel (`hello-runner.yml` exécuté sur `[self-hosted, lab]`)	5

Critère	Points
Action composite `setup-node-project` correcte (inputs, cache sur le bon lockfile, `shell: bash`)	5
Workflow réutilisable `workflow_call` avec inputs, jobs lint → test → build, needs	6
`ci.yml` : deux appels (front + api), `permissions`, `concurrency`, run vert	4
Rapport JUnit publié + couverture résumée + artefacts	3
Branche `main` protégée par les checks requis	2

Erreurs courantes

Le runner reste `Offline` / `Runner successfully added` puis plus rien Le conteneur a été arrêté ou le token était déjà utilisé. `docker compose logs runner` ; si `Http response code: NotFound`, regénérez un token et recréez le conteneur (`docker compose down -v && docker compose up -d`).

`Can't find 'action.yml' ... under '/home/runner/work/.../.github/actions/setup-node-project'` Il manque `actions/checkout@v4` avant l'appel de l'action locale.

`Required property is missing: shell` Dans une action composite, chaque step `run:` doit préciser `shell: bash`.

`Resource not accessible by integration` sur `dorny/test-reporter` `permissions: checks: write` absent (ou déclaré dans le workflow réutilisable au lieu du workflow appelant).

`No test report files were found` Le chemin `path:` ne pointe pas vers `<working-directory>/reports/junit.xml`, ou `npm run test:ci` n'a pas généré le fichier (vérifiez le script dans `package.json`).

`Job 'build' depends on unknown job 'lint'` Faute de frappe dans `needs: [lint, test]` : les noms sont les **identifiants** des jobs, pas leur `name:`.

Ressources

- Documentation — Self-hosted runners
- Documentation — Reusing workflows
- Documentation — Creating a composite action
- Documentation — Using concurrency
- dorny/test-reporter
- Cheatsheet GitHub Actions du cours

Prochain TP : Jour 2 — Déploiement automatisé GitHub Actions + Ansible